PIMPRI CHINCHWAD EDUCATION TRUST's

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

SECTOR NO. 26, NIGDI PRADHIKARN, PUNE – 411044.

A MINI PROJECT REPORT ON

"AI Resume Analyzer"

BACHELOR OF ENGINEERING (TY B. Tech.)

Academic Year: 2025-26

By:

123B1B261 Neelay Shah

Under The Guidance of
Ms. Suvarna Patil

1. Abstract

AI Resume Analyzer is a full-stack web application built on the MERN technology stack (MongoDB, Express.js, React, Node.js) that enables job seekers to upload their resume, receive a comprehensive AI-driven analysis, and obtain a rewritten, ATS-optimized version of their resume — all within a single integrated platform.

The platform accepts PDF and DOCX resumes, extracts text using pdf-parse and Mammoth, and sends the extracted content along with an optional job description to the Gemini AI API (gemini-2.5-flash model). The AI returns a structured JSON analysis covering an overall resume score (0–100), skills gap analysis (matched, missing, and extra skills), ATS compatibility assessment, project quality evaluation, keyword optimization suggestions, experience level classification, recommended job roles, and actionable improvement suggestions.

A dedicated Resume Writer module allows authenticated users to request an AI-generated rewrite of their resume with configurable tone (balanced, concise, executive) and focus (general, ATS, impact, projects). The rewrite output includes a professional headline, summary, key skills, experience bullets, and project bullets — all grounded strictly in the user's original resume content. The application is secured with JWT-based authentication, rate-limited with express-rate-limit, hardened with Helmet, and backed by MongoDB with Mongoose for persistent analysis storage.

2. Introduction

2.1 Background and Motivation

In today's highly competitive job market, a resume must do more than list qualifications — it must pass automated Applicant Tracking Systems (ATS) before a human recruiter ever sees it. Studies suggest that over 75% of resumes are rejected by ATS filters before reaching a hiring manager. Yet most candidates receive no feedback on why their resume failed to progress.

The gap between a candidate's actual qualifications and their ability to present them effectively in an ATS-compatible format is a significant and underserved problem. Existing tools either provide only superficial keyword matching, are expensive SaaS products, or lack the ability to rewrite resume content intelligently. There is a clear need for an open, self-hostable, AI-powered platform that combines deep resume analysis with actionable rewriting capabilities.

2.2 Problem Statement

Job seekers face a multi-faceted problem when applying to roles:

- Resumes are frequently rejected by ATS systems due to formatting issues, missing keywords, or poor structure — even when the candidate is genuinely qualified.
- Candidates receive no structured feedback on what specific skills are missing relative to a target job description.
- Generic resume templates and writing guides cannot adapt to the specific language and requirements of individual job postings.
- Rewriting a resume for each role is time-consuming and cognitively demanding without expert guidance.

- Existing AI resume tools often hallucinate credentials, invent employers, or add fictitious impact numbers — creating ethical and accuracy problems.

2.3 Proposed Solution

AI Resume Analyzer proposes a full-stack MERN application that ingests a user's resume and an optional job description, performs a deep structured analysis using the Gemini AI model, and provides a rewritten resume grounded strictly in the uploaded content. The system is built around a truthful-first AI prompt design that explicitly prohibits the model from inventing credentials, tools, or impact numbers.

3. Aim

The primary aim of AI Resume Analyzer is to design, develop, and deploy an intelligent web application that empowers job seekers to understand the quality of their resume, identify gaps relative to target roles, receive ATS compatibility feedback, and obtain an AI-generated, truthfully-rewritten version of their resume — all through a single, authenticated, full-stack platform.

Specifically, the platform aims to:

- Accept PDF and DOCX resume uploads from authenticated users and reliably extract their full text content for analysis.
- Send extracted resume text and job descriptions to the Gemini AI model and receive a structured JSON analysis covering score, skills, ATS quality, projects, keywords, and role recommendations.
- Persist every analysis result in MongoDB so users can revisit past analyses from their dashboard without re-uploading.
- Provide a Resume Writer module that allows users to configure tone and focus and receive an AI-generated rewrite of their resume content with honest, ATS-optimized language.
- Implement JWT-based user authentication so that analysis history and rewrite results are private and user-specific.
- Apply production-grade security practices including bcrypt password hashing, Helmet HTTP hardening, rate limiting, and CORS origin restriction.

4. Objectives

4.1 Primary Objectives

- Design and implement a secure user authentication system using JWT tokens, bcrypt password hashing, and Mongoose user models with email uniqueness enforcement.
- Build a resume upload pipeline using Multer for file handling, pdf-parse for PDF text extraction, and Mammoth for DOCX text extraction, with a minimum-length validation guard to reject unreadable files.

- Integrate the Gemini AI API (gemini-2.5-flash) with a structured JSON schema response to produce consistent, schema-validated analysis output including score, skills analysis, ATS analysis, project analysis, keyword optimization, experience level, and recommended roles.
- Implement a fallback analysis engine that generates a local heuristic analysis when the Gemini API key is absent or the API call fails, ensuring the application remains functional without API access.
- Persist all analysis results in MongoDB using a comprehensive Mongoose schema that stores the full AI output, matched/missing/extra skills, ATS issues, project feedback, keyword gaps, and recommended roles.
- Build a Resume Writer module that calls the Gemini API with a rewrite-specific schema to produce a headline, professional summary, key skills, experience bullets, project bullets, ATS keywords used, and rewrite notes.
- Create a React frontend with protected routes, a dashboard showing all past analyses, a result page with score visualization and detailed analysis breakdown, and a rewrite workspace.

4.2 Secondary Objectives

- Implement rate limiting (250 requests per 15-minute window) to prevent API abuse and protect both the application and the Gemini API quota.
- Apply Helmet.js HTTP security headers with a cross-origin resource policy configured for file serving from the uploads directory.
- Design a responsive dark/light-mode compatible UI using Tailwind CSS utility classes with a teal/mint accent color scheme appropriate for a professional tool.
- Structure the backend codebase with clean separation between routes, controllers, models, utilities, and middleware for maintainability.
- Provide clear error messages and appropriate HTTP status codes for all failure scenarios including invalid files, missing API keys, database errors, and unauthorized access.

5. Scope of the Project

5.1 In-Scope Features

Module	Feature	Details Included
User Authentication	Register & Login	JWT tokens, bcrypt hashing, email validation, password strength enforcement
Resume Upload	PDF & DOCX ingestion	Multer file handling, pdf-parse + Mammoth text extraction, 5 MB size limit

AI Analysis	Gemini-powered scoring	Score 0–100, skills gap, ATS analysis, project review, keyword optimization, role recommendations
Fallback Analysis	Local heuristic engine	Keyword-based scoring and skills detection when Gemini API is unavailable
Analysis History	Dashboard view	All past analyses listed with score, filename, and date; accessible after login
Result Page	Full analysis breakdown	Score circle, matched/missing/extra skills, ATS issues, project feedback, suggestions, job roles
Resume Writer	AI rewrite generation	Configurable tone and focus; outputs headline, summary, skills, experience bullets, project bullets
Data Persistence	MongoDB storage	Full analysis stored per user; retrievable by ID with enrichment fallback for legacy records
Security	JWT + Helmet + Rate Limit	Protected routes, HTTP hardening, 250 req/15 min rate limiting, CORS origin restriction

5.2 Out-of-Scope

- Real-time collaboration or resume sharing between users.
- Resume parsing into structured sections (education, experience, skills) — raw text extraction is used.
- Direct job application submission or integration with job boards such as LinkedIn or Naukri.
- Native mobile application; only a responsive web interface is provided.
- File format support beyond PDF and DOCX (e.g., RTF, ODT, TXT).
- Version history or diff-based comparison between multiple resume uploads.

6. Technology Stack

AI Resume Analyzer is built on the MERN stack — MongoDB, Express.js, React, and Node.js — extended with the Gemini AI API for resume analysis and rewriting. This stack was selected for its end-to-end JavaScript consistency, strong ecosystem support, and suitability for building AI-integrated RESTful APIs with rich React frontends.

Layer	Technology	Role in Project
M	MongoDB + Mongoose	NoSQL document database storing user accounts and all analysis results. Mongoose provides schema validation, index management, and timestamp support.
E	Express.js	Minimal Node.js web framework defining RESTful API routes (/api/auth, /api/resume). Handles request parsing, file uploads, error middleware, and rate limiting.
R	React (Vite)	Powers all UI views: login/register, dashboard, upload page, result page, and resume writer. Vite provides fast HMR during development. React Router handles client-side routing.
N	Node.js (ESM)	JavaScript runtime executing the Express server and all backend logic. Uses ES module syntax throughout for modern import/export compatibility.

6.1 Supporting Libraries and Services

Library / Service	Role
Gemini AI API (gemini-2.5-flash)	Primary AI engine for structured resume analysis and rewriting via REST API calls with JSON schema response enforcement
pdf-parse	Extracts raw text from uploaded PDF resumes for AI processing
Mammoth	Extracts raw text from uploaded DOCX resumes, converting Word XML to plain text
Multer (v2)	Handles multipart/form-data file uploads; validates MIME type and enforces 5 MB file size limit
jsonwebtoken	Generates and verifies JWT tokens for stateless user authentication

bcryptjs	Hashes user passwords with salt factor 12 before storage; compares passwords on login
Helmet	Sets secure HTTP response headers including cross-origin resource policy for file serving
express-rate-limit	Limits each IP to 250 requests per 15-minute window to prevent abuse
validator	Validates email format and enforces strong password requirements on registration
React Router DOM	Handles client-side navigation, protected routes, and URL-based result loading
Axios	HTTP client used by the React frontend to call the Express API with JWT authorization headers
Lucide React	Icon library providing UI icons throughout the frontend
react-hot-toast	Displays non-blocking success and error notification toasts in the UI
Tailwind CSS	Utility-first CSS framework used for all frontend styling with dark mode support

7. System Architecture

7.1 High-Level Architecture

AI Resume Analyzer follows a three-tier architecture: a React client layer, a Node.js/Express API server layer, and a MongoDB persistence layer. The frontend communicates with the backend exclusively through a REST API secured with JWT bearer tokens. File uploads are handled as multipart/form-data requests. All AI inference is delegated to the external Gemini API from the backend server, keeping API keys and AI logic server-side and out of reach of the client.

7.2 Authentication and Session Model

The application uses a stateless JWT authentication model. On registration, the user's password is hashed with bcrypt (salt factor 12) and stored in MongoDB. On login, the server verifies the password, generates a signed JWT, and returns it to the client. The React frontend stores the token in memory via a custom useAuth hook and attaches it as a Bearer token on all subsequent API requests. The protect middleware on the backend verifies the token on every protected route, extracts the user ID, and attaches the user object to the request. No session state is stored on the server.

7.3 Data Flow — Resume Analysis

- User uploads a PDF or DOCX file and pastes an optional job description on the Upload page.
- The React frontend sends a multipart/form-data POST request to /api/resume/upload with the JWT token in the Authorization header.
- The backend Multer middleware stores the file in the uploads/ directory and validates its MIME type.
- The extractTextFromResume utility reads the file path, selects pdf-parse or Mammoth based on the file extension, and returns normalized plain text.
- The analyzeResume utility sends the resume text and job description to the Gemini API with a strict JSON schema, a low temperature (0.35), and a system prompt that prohibits hallucination.
- The structured AI response is validated, sanitized, and stored in MongoDB as an Analysis document linked to the authenticated user's ID.
- The formatted analysis is returned to the frontend as JSON, and the user is redirected to the result page at /results/:id.

7.4 Data Flow — Resume Rewrite

- From the Result page, the user navigates to the Resume Writer at /writer/:id, selecting tone and focus options.
- The frontend sends a POST request to /api/resume/:id/rewrite with the selected tone and focus in the request body.
- The backend fetches the stored analysis from MongoDB, enriches it with fallback analysis if any fields are missing, and calls the Gemini API with the rewrite-specific schema and prompt.
- The rewrite schema enforces output of: headline, professional_summary, key_skills, experience_bullets, project_bullets, ats_keywords_used, and rewrite_notes.
- The rewrite result is returned to the frontend and displayed in the Resume Writer workspace, where the user can copy individual sections.

7.5 Fallback and Enrichment Strategy

The system includes two defensive mechanisms to ensure consistent output even under degraded conditions. First, if the Gemini API key is absent or the API call fails, the analyzeResume and rewriteResume utilities fall back to a local heuristic engine (buildFallbackAnalysis) that performs keyword-based scoring and skill extraction from the resume text. Second, when a stored Analysis document is missing fields — which can occur for older records saved before schema additions — the getAnalysisById and rewriteAnalysisResume controllers detect the missing fields using needsAnalysisEnrichment and merge in fallback-generated values before responding to the client.

8. Application Features

8.1 User Authentication

The Register screen accepts a display name, email address, and password. The backend validates that the email is well-formed using the validator library and that the password meets minimum strength requirements (at least 8 characters including uppercase, lowercase, and a number). Duplicate email registration is rejected with a 409 status. On success, a JWT is returned and the user is immediately authenticated and redirected to the dashboard.

The Login screen accepts email and password. The backend uses bcrypt to compare the submitted password against the stored hash. Invalid credentials return a 401 status with a generic error message that does not distinguish between wrong email and wrong password, preventing username enumeration.

8.2 Dashboard

The Dashboard displays all of the authenticated user's past resume analyses in reverse chronological order. Each analysis card shows the original filename, the AI score with a visual score indicator, the file type (PDF or DOCX), the date of analysis, and links to view the full result or open the Resume Writer for that analysis. New analyses are initiated via the "Analyze a Resume" button that navigates to the Upload page.

8.3 Resume Upload and Analysis

The Upload page presents a file drop zone accepting PDF and DOCX files up to 5 MB, and a textarea for pasting a target job description. On submission, a loading state is shown with a descriptive message explaining the ongoing AI processing steps. On completion, the user is automatically navigated to the result page.

8.4 Result Page

The Result page is the primary analysis output screen. It is organized into the following sections:

- Score Circle: A circular progress indicator displaying the overall resume score from 0 to 100.
- Summary: A one-paragraph AI-generated summary of the resume's overall quality and fit for the target role.
- Skills Analysis: Three categorized lists showing matched skills (present in both resume and JD), missing skills (required by JD but absent from resume), and extra skills (present in resume but not mentioned in JD).
- ATS Analysis: An ATS compatibility score and a list of specific ATS issues identified (e.g., missing contact info sections, non-standard headings, formatting problems).
- Project Analysis: A project quality score and narrative feedback on the strength and presentation of the candidate's projects.
- Keyword Optimization: A keyword coverage score and a list of missing keywords that should be incorporated into the resume.

- Strengths and Weaknesses: AI-identified positive attributes and areas for improvement specific to the uploaded resume.
- Suggestions: A prioritized list of specific, actionable improvements the candidate should make.
- Recommended Job Roles: A list of job titles the AI identifies as well-matched to the candidate's current resume.

8.5 Resume Writer

The Resume Writer is a configurable AI rewrite workspace. The user selects a tone (Balanced, Concise, or Executive) and a focus area (General, ATS-Optimized, Impact-Focused, or Projects-Focused) before requesting a rewrite. The backend builds a prompt grounded in the stored resume text and analysis, instructing Gemini to rewrite with truthful, ATS-friendly language that never invents credentials or impact numbers.

The rewrite output is displayed in clearly labeled sections: Headline, Professional Summary, Key Skills, Experience Bullets, Project Bullets, ATS Keywords Used, and Rewrite Notes. Each section can be individually copied for use in the candidate's own resume editor.

9. Database Design

9.1 Overview

MongoDB is used as the database layer for its flexible document model, which is well-suited to storing AI-generated analysis output of variable structure. Mongoose provides schema enforcement, index management, and automatic timestamp generation. Two collections handle the application's persistence needs.

9.2 Users Collection

Field	Type and Description
name	String — User's display name. Required, trimmed, max 80 characters.
email	String — User's email address. Required, unique, lowercased, trimmed.
password	String — bcrypt-hashed password. Required, min 8 characters. select: false prevents leakage in queries.
createdAt	Date — Auto-generated by Mongoose timestamps on account creation.
updatedAt	Date — Auto-updated by Mongoose timestamps on every modification.

9.3 Analyses Collection

Field	Type and Description
user	ObjectId — Reference to the User who owns this analysis. Indexed for fast dashboard queries.
originalFileNam e	String — The original uploaded file name, displayed on dashboard and result cards.
storedFileName	String — The Multer-assigned filename in the uploads directory.
fileType	String — Enum: 'pdf' or 'docx'. Determined from the file extension.
resumeText	String — Full extracted plain text of the resume. Required. Used for enrichment and rewriting.
jobDescription	String — Optional target job description submitted by the user. Defaults to empty string.
score	Number — Overall AI-generated resume score from 0 to 100. Required.
matchedSkills	String[] — Skills present in both the resume and the job description.
missingSkills	String[] — Skills required by the JD but absent from the resume.
extraSkills	String[] — Skills on the resume not mentioned in the JD.
suggestions	String[] — Prioritized list of improvement recommendations.
strengths	String[] — Positive attributes identified in the resume.
weaknesses	String[] — Areas needing improvement.
summary	String — One-paragraph AI narrative summary of the resume quality.
skillsAnalysis	Object — Nested object with matched_skills, missing_skills, and extra_skills arrays.
experienceLevel	String — Enum: Beginner, Intermediate, or Advanced.
atsAnalysis	Object — { score: Number, issues: String[] } ATS compatibility evaluation.
projectAnalysis	Object — { score: Number, feedback: String } Project quality evaluation.
keywordOptimiz ation	Object — { score: Number, missing_keywords: String[] } Keyword gap analysis.

recommendedJobRoles	String[] — Job titles the AI recommends based on the resume.
fullAnalysis	Mixed — The complete raw Gemini API response stored for future reference.

9.4 Design Strengths

Storing the full AI response in the fullAnalysis field as a Mixed type provides a safety net for future schema additions — new fields can be backfilled from this stored payload rather than requiring re-analysis. The needsAnalysisEnrichment pattern in the controller detects and fills missing fields at read time, ensuring backward compatibility. All score fields use a 0–100 Number type with min/max constraints, preventing invalid AI outputs from corrupting the database.

10. AI Integration

10.1 Gemini API Architecture

The application integrates with the Google Gemini API using the gemini-2.5-flash model for both analysis and rewriting tasks. API calls are made directly from the Node.js backend using the Fetch API, keeping the API key server-side. Both the analysis and rewrite utilities use generationConfig with responseSchema to enforce structured JSON output from the model — a critical design decision that eliminates the need for fragile output parsing and ensures consistent field presence.

10.2 Analysis Prompt Design

The system prompt for analysis is:

> "You are an advanced ATS (Applicant Tracking System), recruiter, and career expert.
> Return only accurate JSON grounded in the provided resume and job description.
> Never invent tools, projects, impact numbers, employers, or dates."

The analysis schema enforces eleven top-level required fields: score, summary, strengths, weaknesses, skills_analysis, experience_level, ats_analysis, project_analysis, keyword_optimization, suggestions, and recommended_job_roles. The additionalProperties: false constraint at every schema level prevents the model from adding unrequested fields, ensuring the response is always fully processable by the controller.

10.3 Rewrite Prompt Design

The system prompt for rewriting is:

> "You are a senior resume writer and recruiter.
> Rewrite resumes with truthful, ATS-friendly language grounded only in the provided resume and job description."

The rewrite is further constrained by tone (balanced/concise/executive) and focus (general/ats/impact/projects) parameters that are injected into the user-turn prompt, allowing the model to adjust writing style while remaining factually grounded. The rewrite schema enforces seven required fields: headline, professional_summary, key_skills, experience_bullets, project_bullets, ats_keywords_used, and rewrite_notes.

10.4 Retry and Fallback Strategy

Each API call is made with a configurable timeout (default 18 seconds) and retry count (default 1). If the Gemini API key is absent, the analyzeResume and rewriteResume functions skip the API call entirely and return the output of the local fallback engine. If the API call throws an error (network failure, quota exhaustion, schema violation), the catch block returns the fallback analysis. This design ensures the application is always available regardless of Gemini API availability.

11. Frontend Design

11.1 Component Structure

The React frontend is organized around the application's workflow and screen hierarchy.

Component / Page	Description
App.jsx	Root routing component. Redirects unauthenticated users to /login. Protected routes are wrapped in ProtectedRoute.
AppLayout.jsx	Shared layout wrapping all authenticated pages with the sidebar navigation.
Sidebar.jsx	Persistent navigation with links to Dashboard, Upload, and Resume Writer. Shows user name and logout.
Login.jsx / Register.jsx	Authentication screens with form validation, error display, and toast notifications.
Dashboard.jsx	Lists all past analyses as cards with score, filename, date, and quick-action links.
UploadResume.jsx	File upload form with drag-and-drop zone, job description textarea, and loading state.
Result.jsx	Full analysis display with ScoreCircle, skills breakdown, ATS analysis, suggestions, and writer navigation.

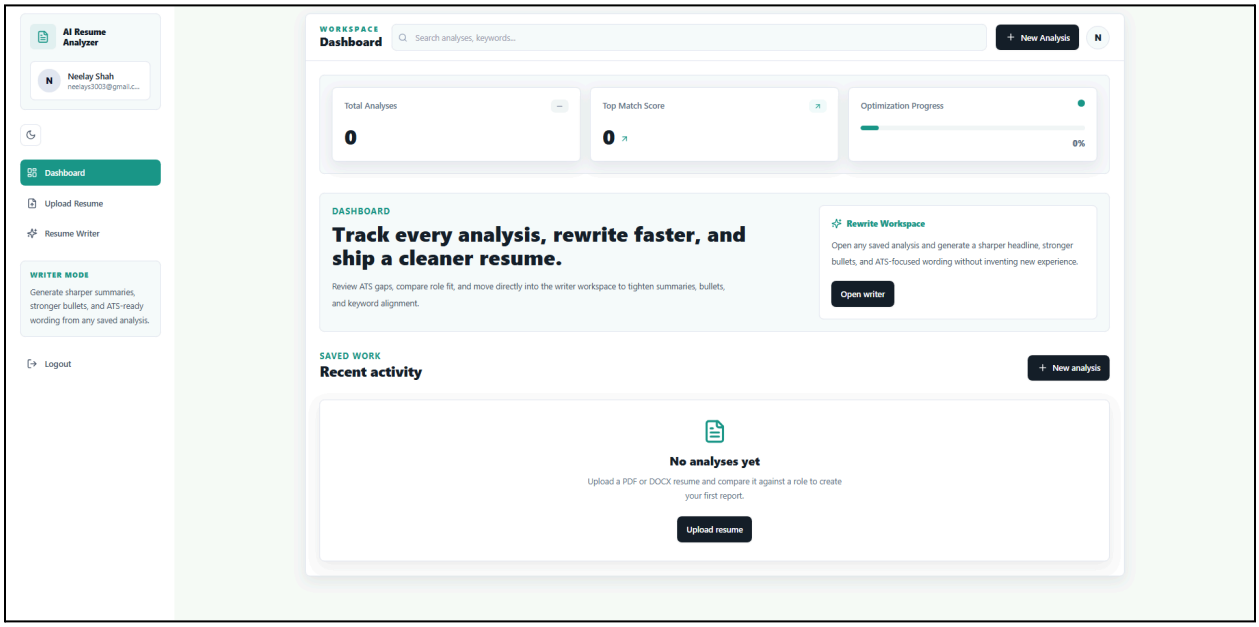
ResumeWriter.jsx	AI rewrite workspace with tone/focus selectors, rewrite trigger, and section-by-section output display.
ScoreCircle.jsx	Circular SVG progress indicator rendering the overall resume score with color-coded zones.
useAuth.jsx	Custom hook managing JWT token state, login/logout actions, and token persistence for the session.

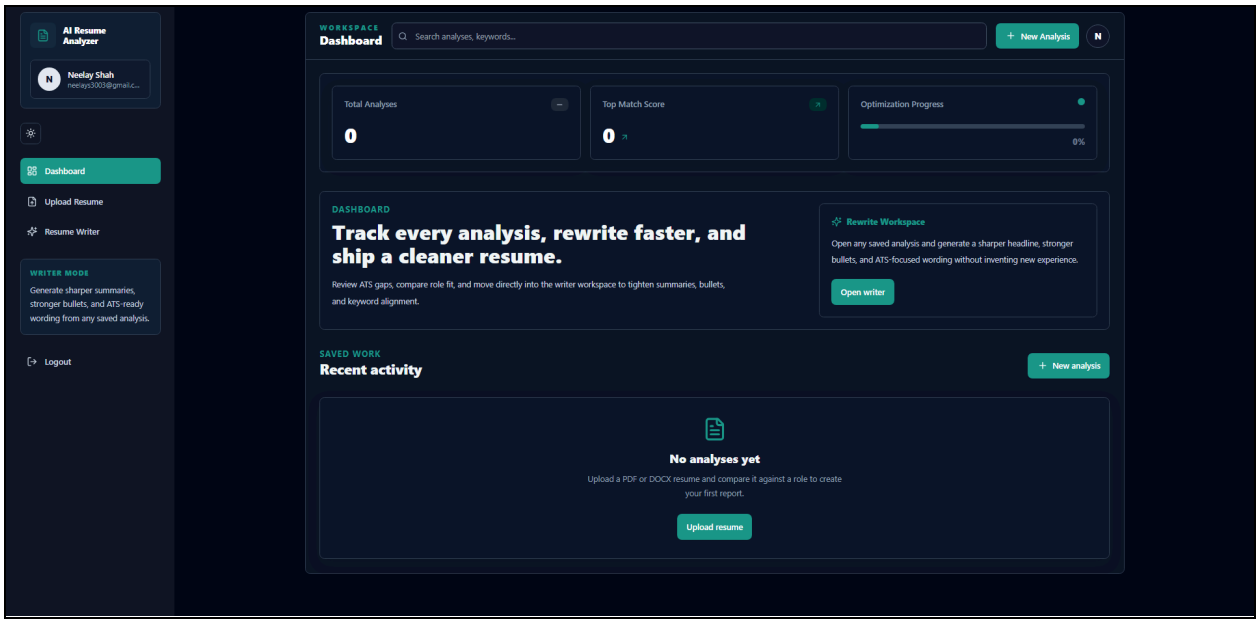
11.2 Routing and Protection

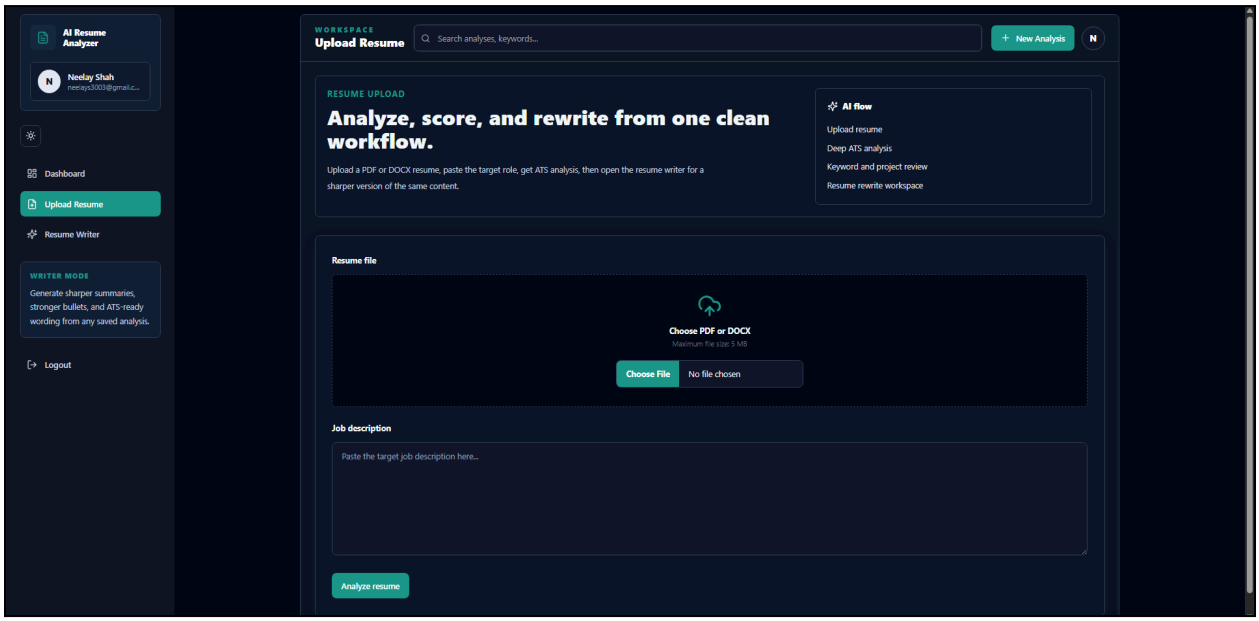
React Router DOM is used for all client-side routing. The ProtectedRoute component wraps all authenticated views and redirects unauthenticated users to /login. The App component also redirects authenticated users away from /login and /register to /dashboard, preventing back-navigation to auth screens after login. The result page (/results/:id) and writer page (/writer/:id) accept analysis IDs as URL parameters, enabling direct deep-linking to any analysis.

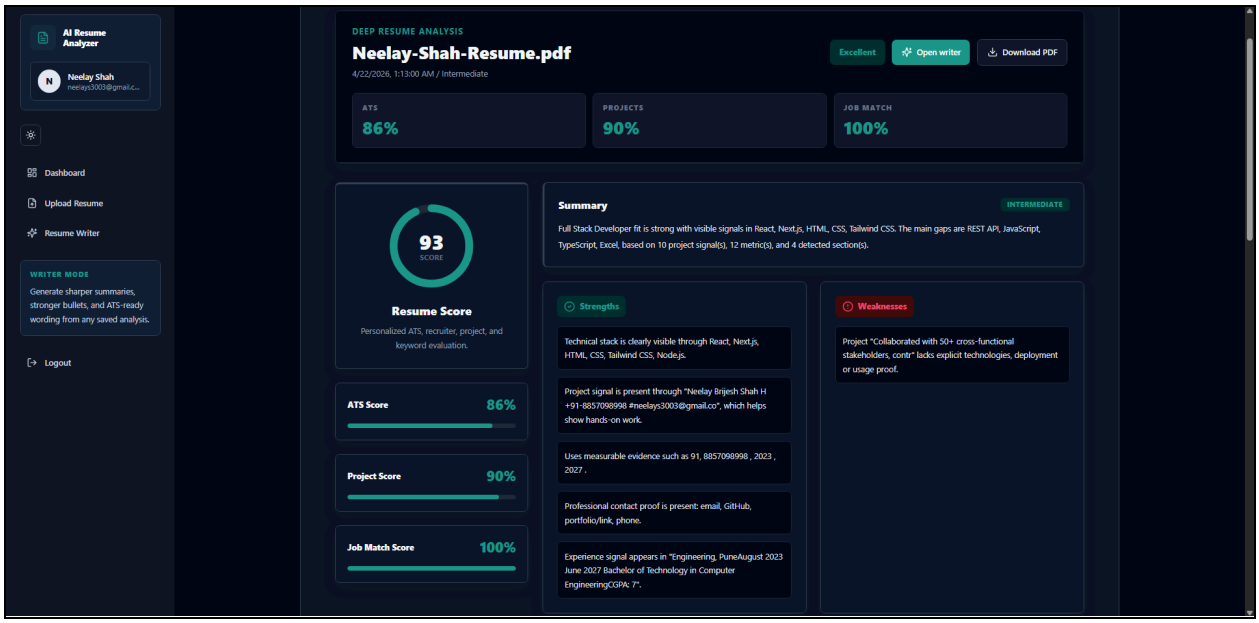
11.3 UI Design Principles

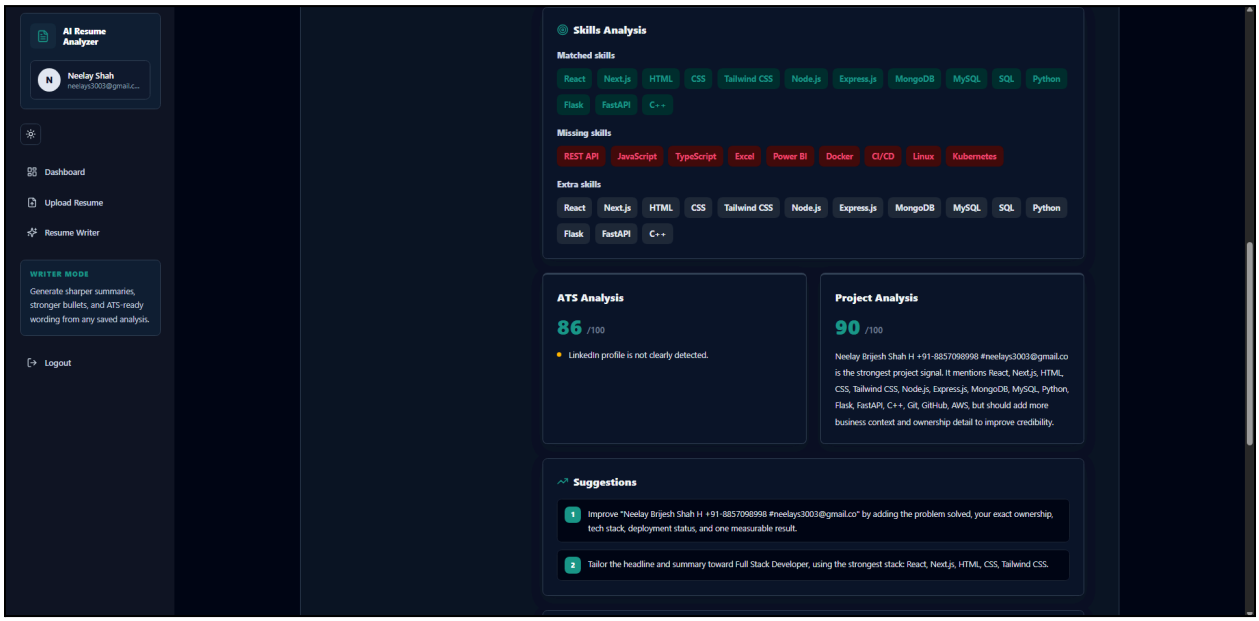
The interface uses a professional, clean aesthetic with a teal/mint accent color scheme applied to primary actions, score indicators, and navigation highlights. A dark mode is supported via Tailwind's dark: variant classes. The layout is responsive with a collapsible sidebar on mobile viewports. All async operations display loading states with descriptive labels, and all API errors are surfaced as react-hot-toast notifications.

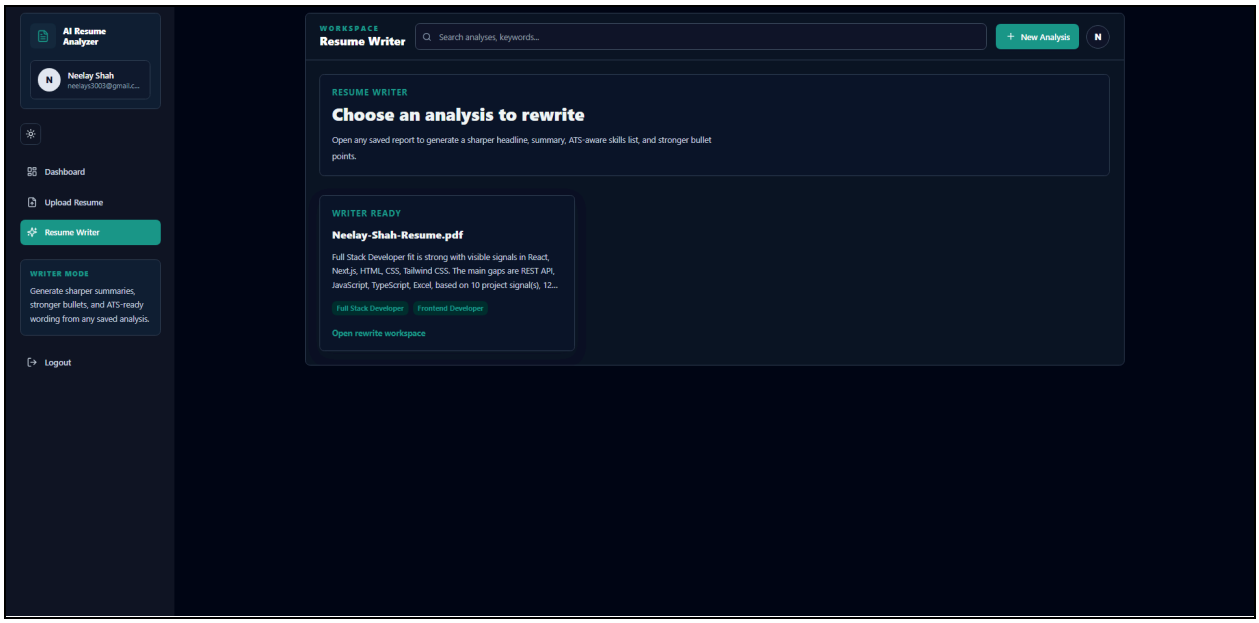


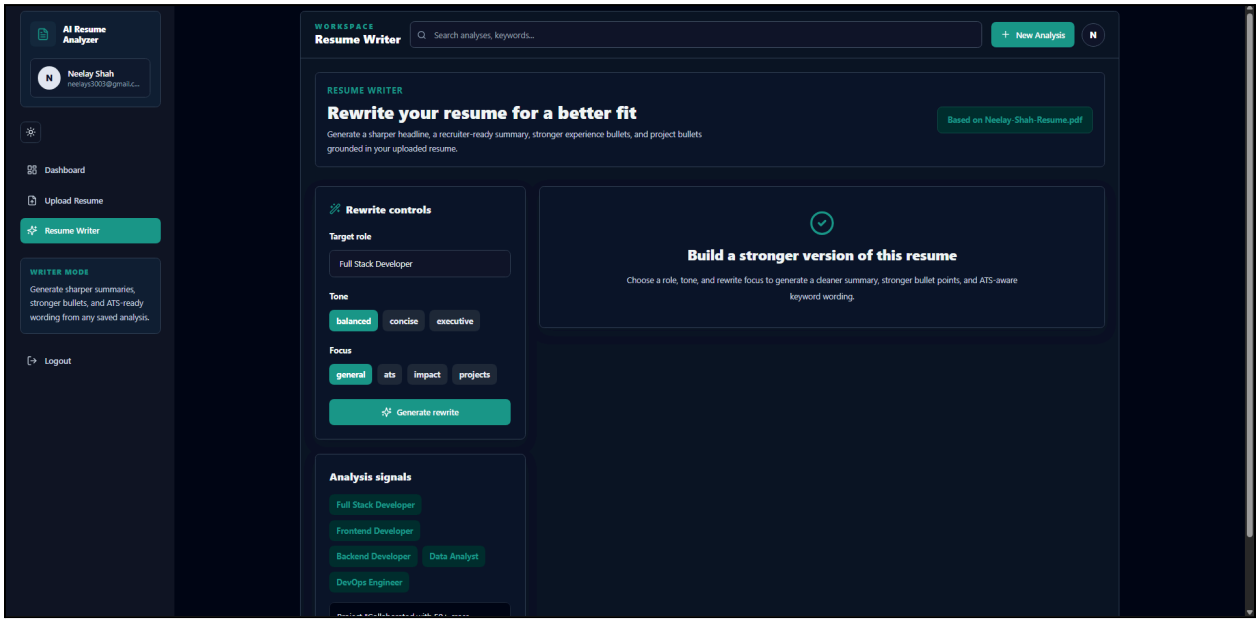

AI Resume Analyzer
Neelay Shah
neelays3003@gmail.c...
Dashboard
Upload Resume
Resume Writer
WRITER MODE
Generate sharper summaries, stronger bullets, and ATS-ready wording from any saved analysis.
Logout
WORKSPACE
Dashboard
Search analyses, keywords...
New Analysis
N
Total Analyses
0
Top Match Score
0
Optimization Progress
0%
DASHBOARD
Track every analysis, rewrite faster, and ship a cleaner resume.
Review ATS gaps, compare role fit, and move directly into the writer workspace to tighten summaries, bullets, and keyword alignment.
Rewrite Workspace
Open any saved analysis and generate a sharper headline, stronger bullets, and ATS-focused wording without inventing new experience.
Open writer
SAVED WORK
Recent activity
New analysis
No analyses yet
Upload a PDF or DOCX resume and compare it against a role to create your first report.
Upload resume

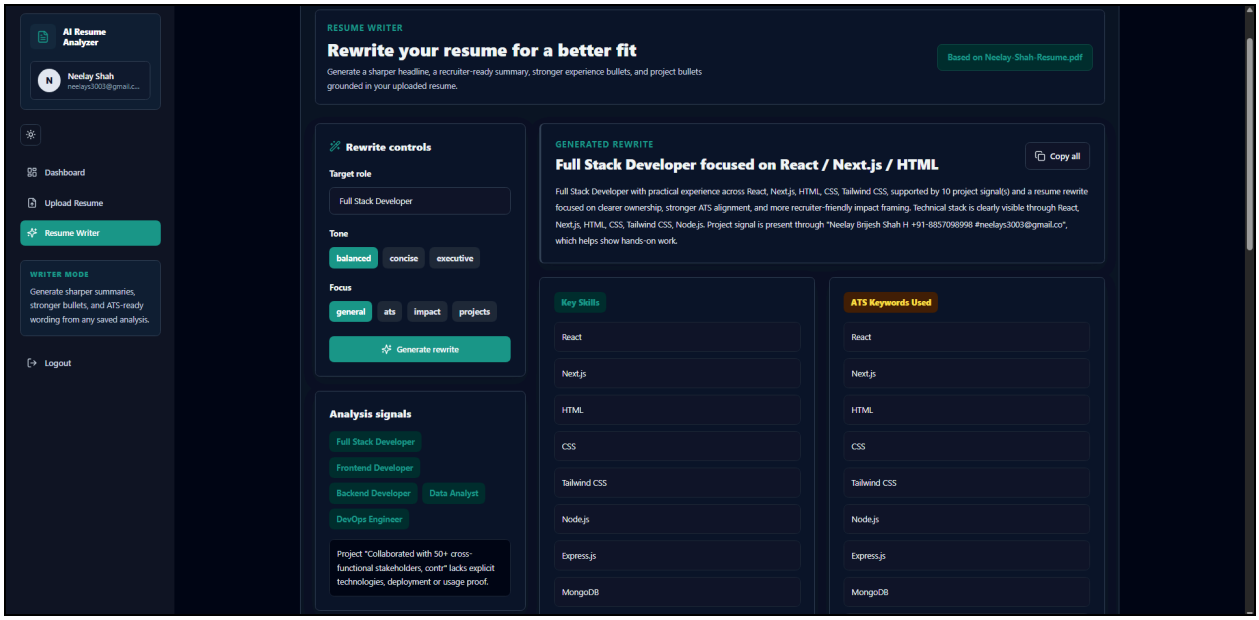

AI Resume Analyzer
Neelay Shah
neelays3003@gmail.c...
Dashboard
Upload Resume
Resume Writer
WRITER MODE
Generate sharper summaries, stronger bullets, and ATS-ready wording from any saved analysis.
Logout
WORKSPACE
Upload Resume
Search analyses, keywords...
New Analysis
N
RESUME UPLOAD
Analyze, score, and rewrite from one clean workflow.
Upload a PDF or DOCX resume, paste the target role, get ATS analysis, then open the resume writer for a sharper version of the same content.
AI flow
Upload resume
Deep ATS analysis
Keyword and project review
Resume rewrite workspace
Resume file
Choose PDF or DOCX
Maximum file size 5 MB
Choose File
No file chosen
Job description
Paste the target job description here...
Analyze resume


AI Resume Analyzer
Neelay Shah
neelays3003@gmail.c...
Dashboard
Upload Resume
Resume Writer
WRITER MODE
Generate sharper summaries, stronger bullets, and ATS-ready wording from any saved analysis.
Logout
DEEP RESUME ANALYSIS
Neelay-Shah-Resume.pdf
4/22/2026, 1:13:00 AM / Intermediate
Excellent
Open writer
Download PDF
ATS
86%
PROJECTS
90%
JOB MATCH
100%
93
SCORE
Resume Score
Personalized ATS, recruiter, project, and keyword evaluation.
ATS Score
86%
Project Score
90%
Job Match Score
100%
Summary
INTERMEDIATE
Full Stack Developer fit is strong with visible signals in React, Next.js, HTML, CSS, Tailwind CSS. The main gaps are REST API, JavaScript, TypeScript, Excel, based on 10 project signal(s), 12 metric(s), and 4 detected section(s).
Strengths
Technical stack is clearly visible through React, Next.js, HTML, CSS, Tailwind CSS, Node.js.
Project signal is present through "Neelay Brijesh Shah H +91-8857098998 #neelays3003@gmail.co", which helps show hands-on work.
Uses measurable evidence such as 91, 8857098998 , 2023 , 2027 .
Professional contact proof is present: email, GitHub, portfolio/link, phone.
Experience signal appears in "Engineering, PuneAugust 2023 June 2027 Bachelor of Technology in Computer EngineeringCGPA: 7".
Weaknesses
Project "Collaborated with 50+ cross-functional stakeholders, contr" lacks explicit technologies, deployment or usage proof.


AI Resume Analyzer
Neelay Shah
neelays3003@gmail.c...
Dashboard
Upload Resume
Resume Writer
WRITER MODE
Generate sharper summaries, stronger bullets, and ATS-ready wording from any saved analysis.
Logout
Skills Analysis
Matched skills
React Next.js HTML CSS Tailwind CSS Node.js Express.js MongoDB MySQL SQL Python Flask FastAPI C++
Missing skills
REST API JavaScript TypeScript Excel Power BI Docker CI/CD Linux Kubernetes
Extra skills
React Next.js HTML CSS Tailwind CSS Node.js Express.js MongoDB MySQL SQL Python Flask FastAPI C++
ATS Analysis
86 /100
LinkedIn profile is not clearly detected.
Project Analysis
90 /100
Neelay Brijesh Shah H +91-8857098998 #neelays3003@gmail.co is the strongest project signal. It mentions React, Next.js, HTML, CSS, Tailwind CSS, Node.js, Express.js, MongoDB, MySQL, Python, Flask, FastAPI, C++, Git, GitHub, AWS, but should add more business context and ownership detail to improve credibility.
Suggestions
1 Improve "Neelay Brijesh Shah H +91-8857098998 #neelays3003@gmail.co" by adding the problem solved, your exact ownership, tech stack, deployment status, and one measurable result.
2 Tailor the headline and summary toward Full Stack Developer, using the strongest stack: React, Next.js, HTML, CSS, Tailwind CSS.


AI Resume Analyzer
Neelay Shah
neelays3003@gmail.c...
Dashboard
Upload Resume
Resume Writer
WRITER MODE
Generate sharper summaries, stronger bullets, and ATS-ready wording from any saved analysis.
Logout
WORKSPACE
Resume Writer
Search analyses, keywords...
New Analysis
N
RESUME WRITER
Choose an analysis to rewrite
Open any saved report to generate a sharper headline, summary, ATS-aware skills list, and stronger bullet points.
WRITER READY
Neelay-Shah-Resume.pdf
Full Stack Developer fit is strong with visible signals in React, Next.js, HTML, CSS, Tailwind CSS. The main gaps are REST API, JavaScript, TypeScript, Excel, based on 10 project signal(s), 12...
Full Stack Developer
Frontend Developer
Open rewrite workspace


AI Resume Analyzer
Neelay Shah
neelays3003@gmail.c...
Dashboard
Upload Resume
Resume Writer
WRITER MODE
Generate sharper summaries, stronger bullets, and ATS-ready wording from any saved analysis.
Logout
WORKSPACE
Resume Writer
Search analyses, keywords...
New Analysis
N
RESUME WRITER
Rewrite your resume for a better fit
Generate a sharper headline, a recruiter-ready summary, stronger experience bullets, and project bullets grounded in your uploaded resume.
Based on Neelay-Shah-Resume.pdf
Rewrite controls
Target role
Full Stack Developer
Tone
balanced
concise
executive
Focus
general
ats
impact
projects
Generate rewrite
Build a stronger version of this resume
Choose a role, tone, and rewrite focus to generate a cleaner summary, stronger bullet points, and ATS-aware keyword wording.
Analysis signals
Full Stack Developer
Frontend Developer
Backend Developer
Data Analyst
DevOps Engineer



12. Security Design

Security Concern	Mechanism Applied
Password storage	bcrypt with salt factor 12 — passwords are never stored in plaintext
API authentication	JWT bearer tokens signed with a strong secret; verified on every protected route
HTTP header hardening	Helmet.js sets X-Content-Type-Options, X-Frame-Options, and other security headers
Cross-origin access	CORS configured to allow requests only from the configured CLIENT_URL origin
File upload abuse	Multer enforces 5 MB file size limit and MIME type restriction to PDF/DOCX only
API abuse	express-rate-limit restricts each IP to 250 requests per 15-minute window
Input validation	Email format and password strength validated server-side using the validator library
Error information leak	Generic error messages on auth failures to prevent username enumeration
Environment secrets	API keys and JWT secret stored in .env; never committed to source control (.gitignore)

13. Configuration and Deployment

13.1 Environment Configuration

All configurable values are managed through environment variables loaded with dotenv. The backend requires the following variables:

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/ai_resume_analyzer
JWT_SECRET=<strong_random_secret>
GEMINI_API_KEY=<gemini_api_key>
GEMINI_MODEL=gemini-2.5-flash
CLIENT_URL=http://localhost:5173
```

The React frontend requires one variable:

```
VITE_API_URL=http://localhost:5000
```

13.2 Deployment Checklist

- backend/.env created with valid MONGO_URI, JWT_SECRET, GEMINI_API_KEY, and CLIENT_URL.
- MongoDB running on port 27017 (local instance or MongoDB Atlas connection string).
- Port 5000 available for the Express server.
- frontend/.env configured with VITE_API_URL pointing to the backend origin.
- npm install run in both backend/ and frontend/ directories.
- npm run dev in backend/ and frontend/ to start the development servers.
- For production: npm run build in frontend/ to generate the dist/ bundle; serve static files or deploy to a hosting platform.

13.3 Error Handling Strategy

Error	Cause	Response
Missing resume file	No file attached to upload request	400 — 'Resume file is required'
Unreadable resume	Extracted text under 80 characters	400 — 'Could not extract enough text from this resume'
Invalid file type	Non-PDF/DOCX file uploaded	400 — Multer rejects the file before the controller runs

Unauthorized access	Missing or invalid JWT token	401 — 'Not authorized, token failed'
Analysis not found	Invalid or another user's analysis ID	404 — 'Analysis not found'
Gemini API failure	Network error or quota exhaustion	Fallback analysis returned; no error exposed to client
MongoDB connection failure	MONGO_URI missing or database down	500 — process.exit(1) on startup; connection error logged
Rate limit exceeded	More than 250 requests in 15 minutes from one IP	429 — Rate limit headers included in response

14. Future Enhancements

Phase 1 — Next Sprint

- LinkedIn profile URL import as an alternative to file upload, parsing the profile page into resume text.
- Resume section parser to identify and separately analyze Education, Experience, Skills, and Projects sections.
- PDF export of the AI-generated rewrite as a formatted, downloadable resume file.

Phase 2 — 2 to 3 Sprints

- Multi-job-description comparison: analyze the same resume against multiple JDs simultaneously and rank fit scores.
- Resume version management: store multiple resume versions per user and allow side-by-side score comparison.
- Email notifications when analysis is complete for large files that take longer to process.
- Admin dashboard for usage analytics, API quota monitoring, and user management.

Phase 3 — Roadmap

- Browser extension to analyze a resume directly against a job posting on LinkedIn or Glassdoor.
- Team/recruiter plan: organizations can invite candidates to submit resumes for AI screening before human review.
- Fine-tuned model: replace the general Gemini model with a domain-specific model fine-tuned on a curated resume dataset for higher precision scoring.
- Real-time collaborative resume editor: allow candidates and career coaches to edit the AI rewrite together in the browser.

15. Conclusion

AI Resume Analyzer successfully demonstrates how the MERN technology stack, combined with the Gemini AI API, can be applied to build a practical, production-oriented tool that addresses a real and widely felt problem in the job-seeking process. The project achieves its stated aim of providing a unified platform where authenticated users can upload their resume, receive a deep structured analysis with ATS compatibility evaluation, skills gap identification, and keyword optimization feedback, and obtain an AI-generated, truthfully-rewritten version of their resume tailored to a target role.

The truthfulness-first AI prompt design — which explicitly prohibits the model from inventing credentials, tools, or impact numbers, and enforces a strict JSON schema at the API level — represents the most significant engineering decision of the project. This design ensures that the AI output is both structurally consistent and ethically sound, avoiding the hallucination problem that undermines trust in many AI resume tools.

The layered fallback and enrichment architecture, which includes a local heuristic analysis engine, a Gemini API retry mechanism, and a read-time field enrichment pattern for legacy records, ensures the application remains functional and responsive under degraded conditions. The security design — combining JWT authentication, bcrypt password hashing, Helmet HTTP hardening, rate limiting, and CORS origin restriction — meets production-grade standards for a user-facing data application.

Future work may include LinkedIn import, PDF resume export from rewrite output, multi-JD comparison, and a fine-tuned domain-specific AI model. The codebase is structured for extensibility with clean separation between routes, controllers, models, utilities, and middleware — facilitating the addition of new features with minimal refactoring.

AI Resume Analyzer is a complete, deployable, AI-powered resume analysis and rewriting platform built on the MERN stack — ready for academic submission, further development, or live deployment.

16. References

1. Google Gemini API Documentation — https://ai.google.dev/docs
2. MongoDB Mongoose ODM Documentation — https://mongoosejs.com/docs/
3. React Documentation — https://react.dev/
4. Express.js Documentation — https://expressjs.com/
5. Vite Build Tool Documentation — https://vitejs.dev/guide/
6. Multer File Upload Middleware — https://github.com/expressjs/multer
7. pdf-parse npm Package — https://www.npmjs.com/package/pdf-parse
8. Mammoth DOCX Extractor — https://github.com/mwilliamson/mammoth.js
9. jsonwebtoken npm Package — https://www.npmjs.com/package/jsonwebtoken
10. Helmet HTTP Security Headers — https://helmetjs.github.io/
11. React Router DOM Documentation — https://reactrouter.com/
12. Tailwind CSS Documentation — https://tailwindcss.com/docs